

Java_Script_WEEK2_Day – 3_Hands On - Gurunath Rageni

LEVEL – 1 (Problem 1): Interactive Feedback Button

Scenario:

A startup company wants a simple interactive webpage where users can click a button to submit feedback and instantly see a confirmation message without refreshing the page.

Requirements

- Create a webpage with a “Submit Feedback” button.
- When clicked, display a confirmation message dynamically.
- Use JavaScript event listeners to handle the click event.

Technical Constraints

- Use plain HTML, CSS, and JavaScript.
- Event handling must use `addEventListener()`.
- No external frameworks allowed.

Expectations:

- Proper DOM manipulation.
- Clean and readable code.

Code:

```
<!DOCTYPE html>
<html>
  <head>
    <title>feedback</title><link rel="styleSheet" href="style.css"></head>
  <body>
    <div class="container">
      <h2>Feedback</h2>
```

```

        <textarea id="feedbackText" rows="2" cols="30" placeholder="Enter feedback
here..."></textarea>

        <br><br>

        <button id="fbtn">Submit Feedback</button>

        <p id="message" class="msg"></p>
    </div>

    <script>

        const button = document.getElementById("fbtn");

        button.addEventListener('click', function() {

            const textBox = document.getElementById("feedbackText");

            const message=document.getElementById("message")

            if (textBox.value.trim() === "") {

                document.getElementById("message").innerText = "Please enter feedback";

                message.style.color='red';

                return;}

            document.getElementById("message").innerText = "Thanks for your feedback";

            message.style.color='green';

            textBox.value = "";

            button.disabled = true;});

    </script>

</body>

</html>

body{

    display: flex;

    justify-content: center;

}

.container{

    text-align: center;

    border: 1px solid;

    border-color: brown;

```

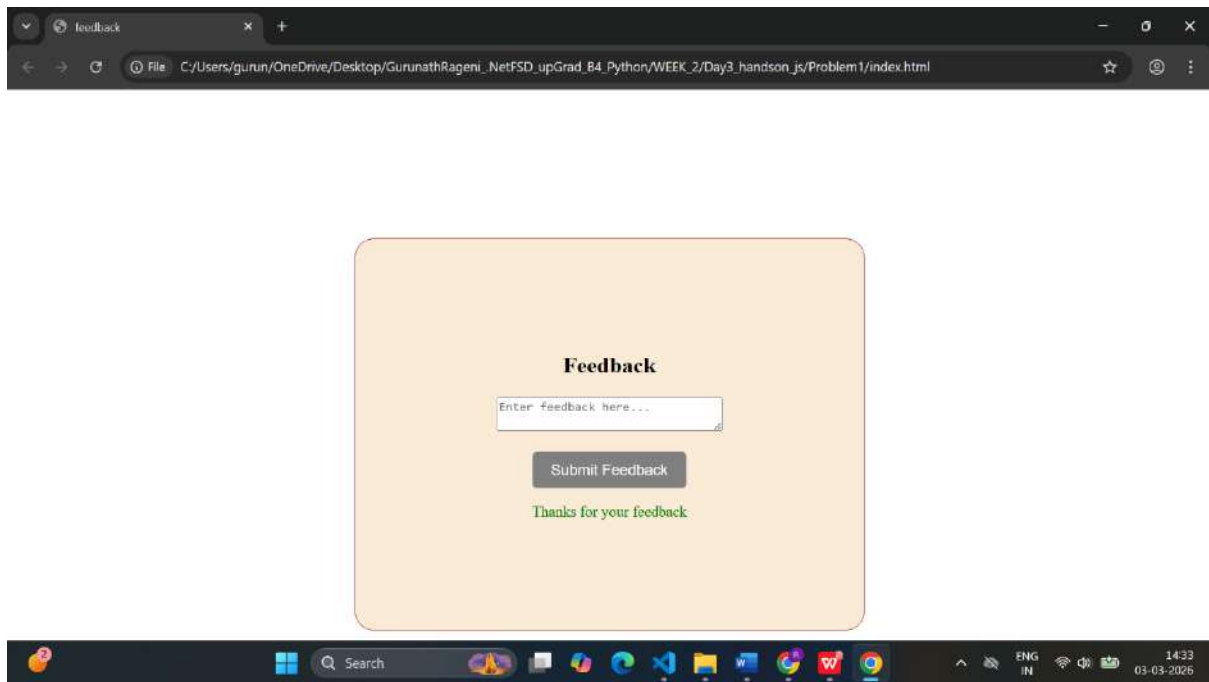
```
padding: 100px 150px;
background-color: antiquewhite;
border-radius: 20px;
margin-top: 150px;
}
button {
padding: 10px 20px;
font-size: 16px;
background-color: #007bff;
color: white;
border: none;
border-radius: 5px;
cursor: pointer;
}
.msg{
color: green;
}
#fbtn:disabled {
background-color: gray;
cursor: not-allowed;
}
```

Code Screenshot:

```
index.html M X
WEEK_2 > Day3_handson_js > Problem1 > index.html > html > head
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>feedback</title><link rel="stylesheet" href="style.css"></head>
5   <body>
6     <div class="container">
7       <h2>Feedback</h2>
8       <textarea id="feedbackText" rows="2" cols="30" placeholder="Enter feedback here..."></text
9       <br><br>
10      <button id="fbtn">Submit Feedback</button>
11      <p id="message" class="msg"></p>
12    </div>
13    <script>
14      const button = document.getElementById("fbtn");
15      button.addEventListener('click', function() {
16        const textBox = document.getElementById("feedbackText");
17        const message=document.getElementById("message")
18
19        if (textBox.value.trim() === "") {
20          document.getElementById("message").innerText = "Please enter feedback";
21          message.style.color='red';
22          return;}
23        document.getElementById("message").innerText = "Thanks for your feedback";
24        message.style.color='green';
25        textBox.value = "";
26        button.disabled = true;});
27    </script>
28  </body>
29 </html>
```

```
<> index.html M    # style.css M X
WEEK_2 > Day3_handson_js > Problem1 > # style.css > .container
1  body{
2      display: flex;
3      justify-content: center;
4  }
5  .container{
6      text-align: center;
7      border: 1px solid;
8      border-color: #800000 brown;
9      padding: 100px 150px;
10     background-color: #FAFAD2 antiquewhite;
11     border-radius: 20px;
12     margin-top: 150px;
13 }
14 button {
15     padding: 10px 20px;
16     font-size: 16px;
17     background-color: #007bff;
18     color: white;
19     border: none;
20     border-radius: 5px;
21     cursor: pointer;
22 }
23 .msg{
24     color: green;
25 }
26 #fbtn:disabled {
27     background-color: gray;
28     cursor: not-allowed;
29 }
```

Output Screenshot:



Explanation:

This code implements client-side form validation using JavaScript by checking whether the textarea input is empty before submission. It dynamically updates the DOM using innerText and modifies styles to display validation messages in red or green. An event listener is attached to the button to handle user interaction, clear the input field after successful submission, and disable the button to prevent multiple submissions. The CSS enhances UI presentation using Flexbox for centering, custom styling, and a :disabled selector to visually indicate button state changes.

LEVEL – 1 (Problem 2): Theme Toggle Application

Scenario:

A small blog website wants to allow users to switch between Light Mode and Dark Mode interactively.

📌 Requirements

- Create a toggle button to switch themes.
- Change background and text color dynamically.
- Store theme preference in localStorage.

🔧 Technical Constraints

- Use Vanilla JavaScript.
- Use `addEventListener()` for events.
- No CSS frameworks.

Expectations:

- Smooth theme switching.
- Proper usage of `localStorage`.
- Clean and modular JavaScript functions.

Code:

```
<!DOCTYPE html>

<html>

<head>

  <title>Theme Toggle App</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

  <div class="container">

    <h1>My Blog Website</h1>

    <p>This is a simple theme toggle application.</p>

    <button id="themeToggleBtn">Switch Theme</button>

  </div>

  <script>

    const toggleBtn = document.getElementById("themeToggleBtn");

    const body = document.body;

    function loadTheme() {

      const savedTheme = localStorage.getItem("theme");
```

```

    if (savedTheme) {
        body.classList.add(savedTheme);
    } else {
        body.classList.add("light-mode");
    }
}

// Toggle theme function
function toggleTheme() {
    if (body.classList.contains("light-mode")) {
        body.classList.remove("light-mode");
        body.classList.add("dark-mode");
        localStorage.setItem("theme", "dark-mode");
    } else {
        body.classList.remove("dark-mode");
        body.classList.add("light-mode");
        localStorage.setItem("theme", "light-mode");
    }
}

// Event Listener
toggleBtn.addEventListener("click", toggleTheme);

// Load theme when page opens
loadTheme();
</script>

</body>
</html>

/* Smooth transition */
body{

```



```
margin: 0;
padding: 0;
font-family: Arial, sans-serif;
transition: background-color 0.3s ease, color 0.3s ease;
}
```

```
/* Light Mode */
```

```
.light-mode {
  background-color: #ffffff;
  color: #000000;
}
```

```
/* Dark Mode */
```

```
.dark-mode {
  background-color: #1e1e1e;
  color: #ffffff;
}
```

```
/* Center content */
```

```
.container {
  text-align: center;
  margin-top: 100px;
}
```

```
button {
  padding: 10px 20px;
  font-size: 16px;
  cursor: pointer;
}
```

Code Screenshot:

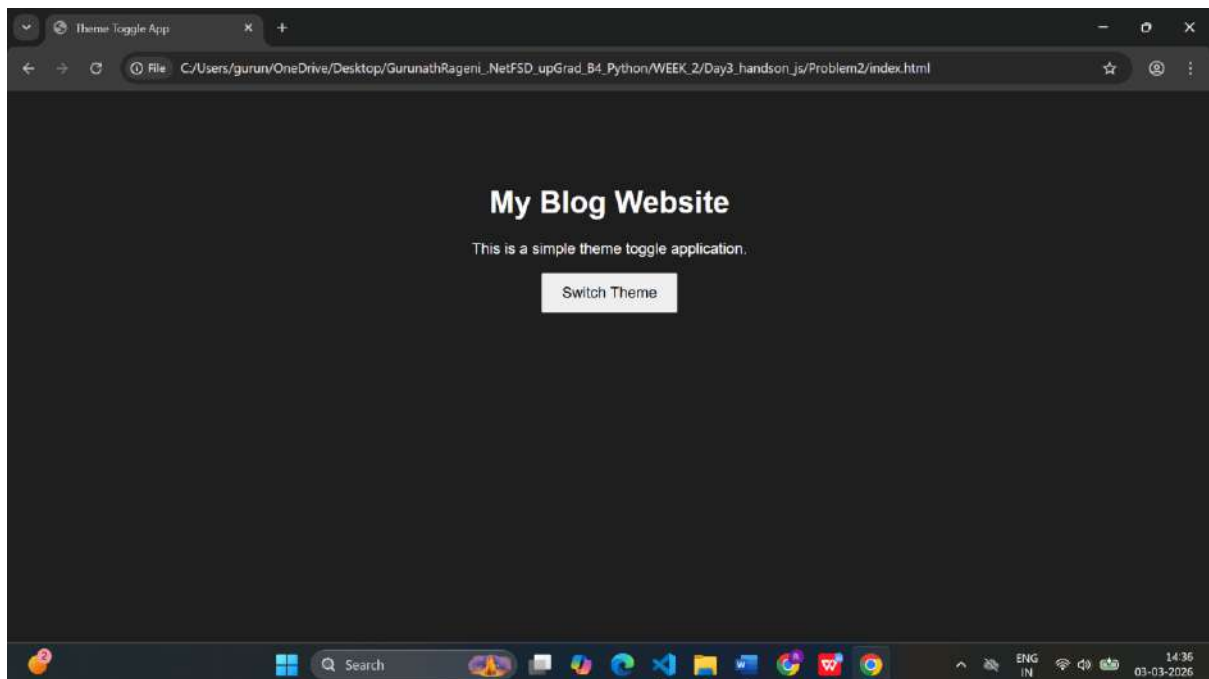
```
<> index.html ...\Problem1 M    <> index.html ...\Problem2 X    # style.css M
WEEK_2 > Day3_handson_js > Problem2 > <> index.html > html > body > script > toggleTheme
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Theme Toggle App</title>
5      <link rel="stylesheet" href="style.css">
6  </head>
7  <body>
8
9      <div class="container">
10         <h1>My Blog Website</h1>
11         <p>This is a simple theme toggle application.</p>
12         <button id="themeToggleBtn">Switch Theme</button>
13     </div>
14     <script>
15         const toggleBtn = document.getElementById("themeToggleBtn");
16         const body = document.body;
17
18         function loadTheme() {
19             const savedTheme = localStorage.getItem("theme");
20
21             if (savedTheme) {
22                 body.classList.add(savedTheme);
23             } else {
24                 body.classList.add("light-mode");
25             }
26         }
27
28         // Toggle theme function
29         function toggleTheme() {
```

```

28     // Toggle theme function
29     function toggleTheme() {
30         if (body.classList.contains("light-mode")) {
31             body.classList.remove("light-mode");
32             body.classList.add("dark-mode");
33             localStorage.setItem("theme", "dark-mode");
34         } else {
35             body.classList.remove("dark-mode");
36             body.classList.add("light-mode");
37             localStorage.setItem("theme", "light-mode");
38         }
39     }
40
41     // Event Listener
42     toggleBtn.addEventListener("click", toggleTheme);
43
44     // Load theme when page opens
45     loadTheme();
46 </script>
47
48 </body>
49 </html>

```

Code output:



Explanation:

This application implements a theme toggle feature using JavaScript by dynamically adding and removing CSS classes (light-mode and dark-mode) on the body element. It uses localStorage to persist the selected theme, ensuring the user's preference is maintained even after page reload. An event listener handles button clicks to switch themes, while the loadTheme() function initializes the saved theme on page load. CSS transitions enhance user experience by providing smooth background and text color changes between modes.

LEVEL – 2 (Problem 3): Dynamic To-Do List

Scenario:

A productivity application requires an interactive To-Do list where users can add, delete, and mark tasks as complete without refreshing the page.

Requirements

- Add new tasks dynamically.
- Delete tasks on button click.
- Mark tasks as completed.
- Use event delegation for better performance.

Technical Constraints

- Must use DOM manipulation.
- Event delegation required.
- Code should follow modular structure.

Expectations:

- Fully functional interactive UI.
- Organized and reusable functions.

Code:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Dynamic To-Do List</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

  <div class="container">

    <h2>My To-Do List</h2>

    <div class="input-section">

      <input type="text" id="taskInput" placeholder="Enter a new task">

      <button id="addBtn">Add Task</button>

    </div>

    <ul id="taskList"></ul>

  </div>

  <script src="script.js"></script>

</body>

</html>
```

```
// Select Elements

const taskInput = document.getElementById("taskInput");
const addBtn = document.getElementById("addBtn");
const taskList = document.getElementById("taskList");

// Add Task

function addTask() {

  const taskText = taskInput.value.trim();

  if (taskText === "") {

    alert("Please enter a task!");
```

```

    return;
}

const li = document.createElement("li");

li.innerHTML = `
    <span class="task-text">${taskText}</span>
    <div class="btn-group">
        <button class="complete-btn">Done</button>
        <button class="delete-btn">Delete</button>
    </div>
`;

taskList.appendChild(li);
taskInput.value = "";
}

// Event Delegation (FIXED)
function handleTaskActions(e) {

    const deleteBtn = e.target.closest(".delete-btn");
    const completeBtn = e.target.closest(".complete-btn");

    // Delete
    if (deleteBtn) {
        deleteBtn.closest("li").remove();
    }

    // Complete
    if (completeBtn) {
        completeBtn.closest("li").classList.toggle("completed");
    }
}

```

```
}  
}
```

```
// Event Listeners
```

```
addBtn.addEventListener("click", addTask);
```

```
taskList.addEventListener("click", handleTaskActions);
```

```
body {  
  font-family: Arial, sans-serif;  
  background-color: #f4f4f4;  
  display: flex;  
  justify-content: center;  
  margin-top: 50px;  
}
```

```
.container {  
  background: white;  
  padding: 20px;  
  width: 350px;  
  border-radius: 8px;  
  box-shadow: 0 0 10px rgba(0,0,0,0.1);  
}
```

```
.input-section {  
  display: flex;  
  gap: 10px;  
}
```

```
input {  
  flex: 1;  
  padding: 8px;
```

```
}
```

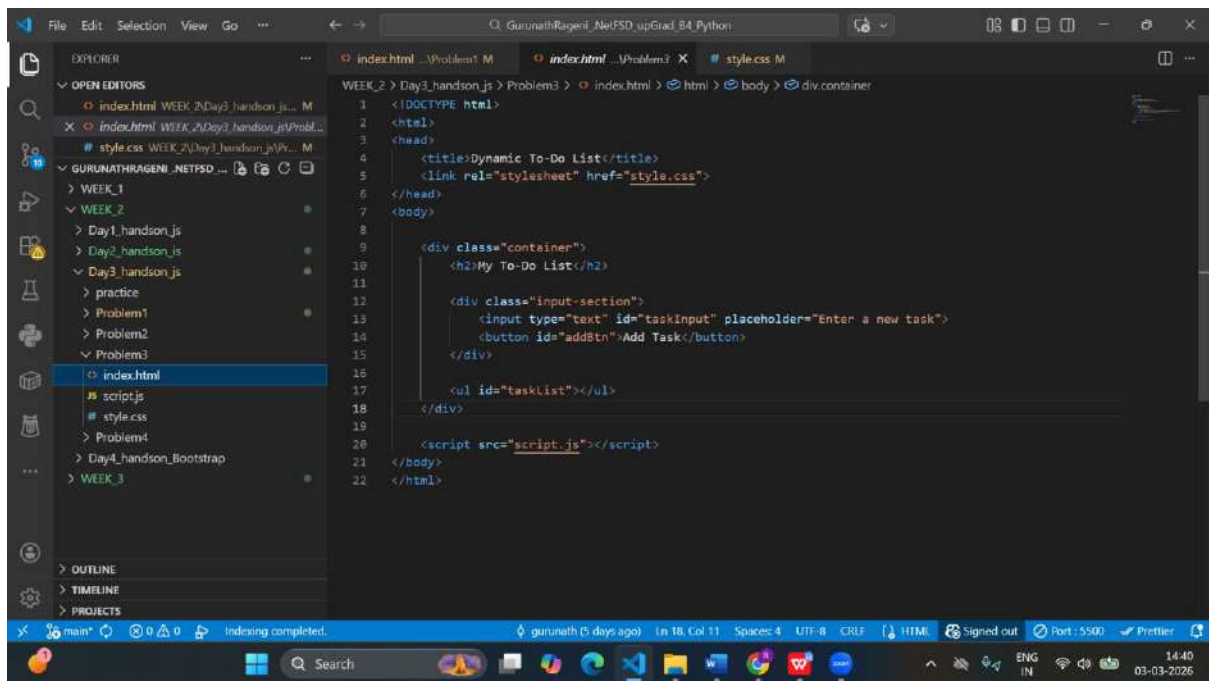
```
button{  
  padding: 8px 12px;  
  cursor: pointer;  
}
```

```
ul{  
  list-style: none;  
  padding: 0;  
  margin-top: 20px;  
}
```

```
li{  
  display: flex;  
  justify-content: space-between;  
  padding: 8px;  
  margin-bottom: 8px;  
  background: #eee;  
  border-radius: 5px;  
}
```

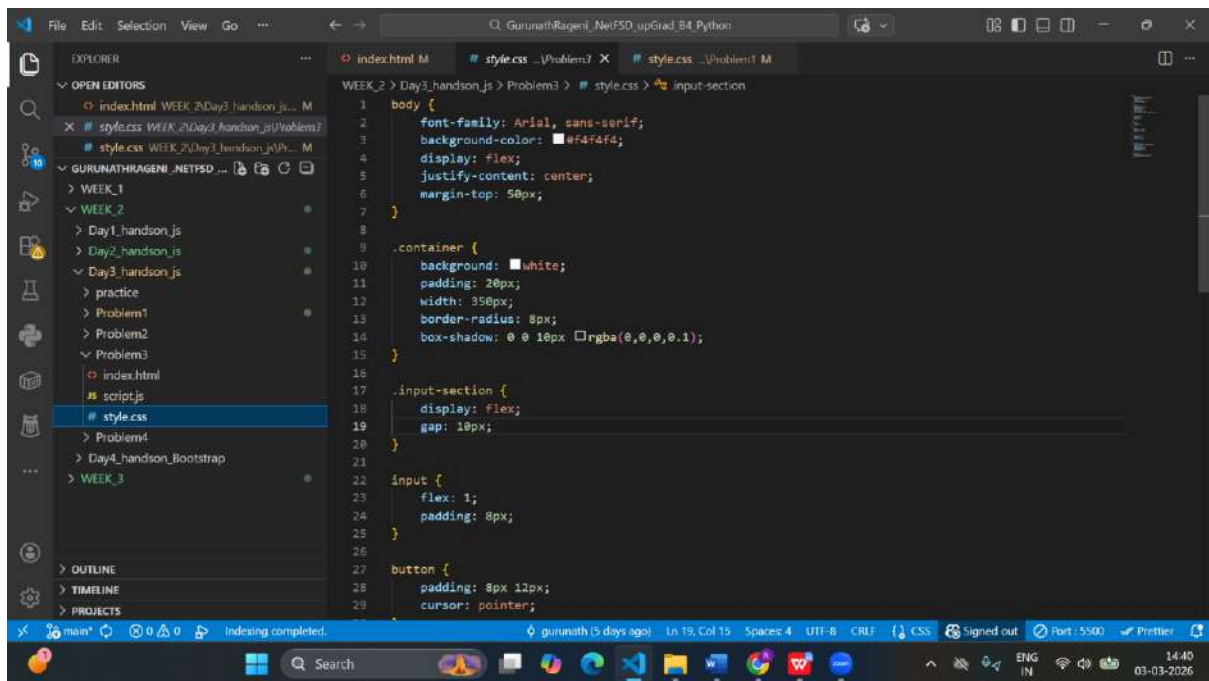
```
.completed{  
  text-decoration: line-through;  
  background-color: #c8e6c9;  
}
```


Code Screenshots:



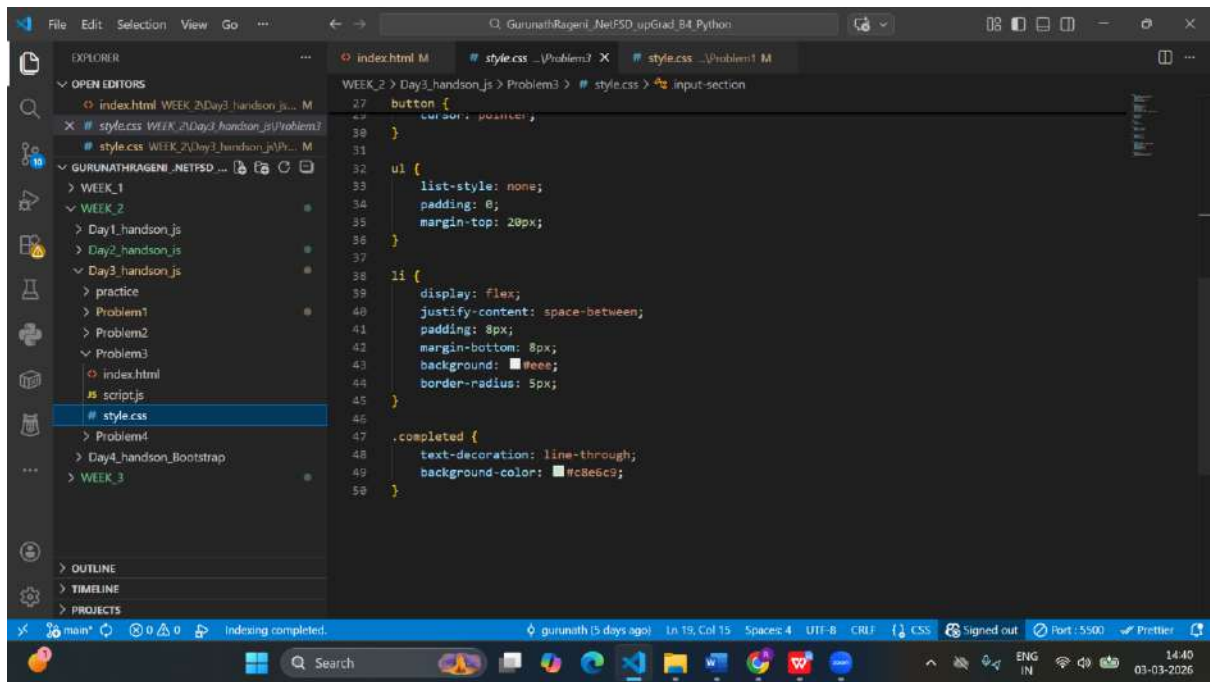
This screenshot shows the VS Code editor with the 'index.html' file open. The Explorer sidebar on the left shows the project structure, including 'WEEK_2' and 'Problem3'. The main editor area displays the following HTML code:

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Dynamic To-Do List</title>
5   <link rel="stylesheet" href="style.css">
6 </head>
7 <body>
8
9   <div class="container">
10     <h2>My To-Do List</h2>
11
12     <div class="input-section">
13       <input type="text" id="taskInput" placeholder="Enter a new task">
14       <button id="addBtn">Add Task</button>
15     </div>
16
17     <ul id="taskList"></ul>
18   </div>
19
20   <script src="script.js"></script>
21 </body>
22 </html>
```



This screenshot shows the VS Code editor with the 'style.css' file open. The Explorer sidebar on the left shows the project structure, including 'WEEK_2' and 'Problem3'. The main editor area displays the following CSS code:

```
1 body {
2   font-family: Arial, sans-serif;
3   background-color: #f4f4f4;
4   display: flex;
5   justify-content: center;
6   margin-top: 50px;
7 }
8
9 .container {
10   background: white;
11   padding: 20px;
12   width: 350px;
13   border-radius: 8px;
14   box-shadow: 0 0 10px rgba(0,0,0,0.1);
15 }
16
17 .input-section {
18   display: flex;
19   gap: 10px;
20 }
21
22 input {
23   flex: 1;
24   padding: 8px;
25 }
26
27 button {
28   padding: 8px 12px;
29   cursor: pointer;
```

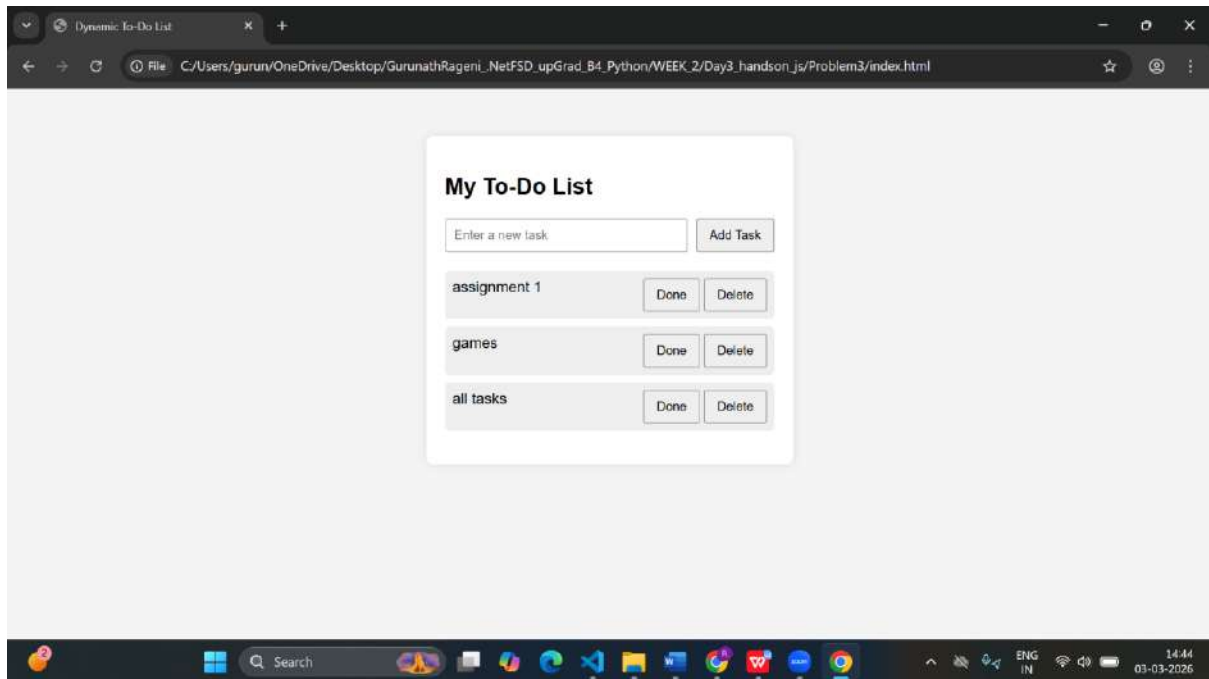


```
1 // Select Elements
2 const taskInput = document.getElementById('taskInput');
3 const addBtn = document.getElementById('addBtn');
4 const taskList = document.getElementById('taskList');
5
6
7 // Add Task
8 function addTask() {
9   const taskText = taskInput.value.trim();
10
11   if (taskText === '') {
12     alert('Please enter a task!');
13     return;
14   }
15
16   const li = document.createElement('li');
17
18   li.innerHTML = `
19     <span class='task-text'>${taskText}</span>
20     <div class='btn-group'>
21       <button class='complete-btn'>Done</button>
22       <button class='delete-btn'>Delete</button>
23     </div>
24   `;
25
26   taskList.appendChild(li);
27   taskInput.value = '';
28 }
29
```

```
29
30
31 // Event Delegation (FIXED)
32 function handleTaskActions(e) {
33
34   const deleteBtn = e.target.closest('.delete-btn');
35   const completeBtn = e.target.closest('.complete-btn');
36
37   // Delete
38   if (deleteBtn) {
39     deleteBtn.closest('li').remove();
40   }
41
42   // Complete
43   if (completeBtn) {
44     completeBtn.closest('li').classList.toggle('completed');
45   }
46 }
47
48
49 // Event Listeners
50 addBtn.addEventListener('click', addTask);
51 taskList.addEventListener('click', handleTaskActions);

```

Output Screenshot:



Explanation:

This application demonstrates dynamic DOM manipulation by creating and appending `<li>` elements when a new task is added. It validates user input, clears the input field after insertion, and structures task items using template literals. Event delegation is implemented by attaching a single click listener to the `<ul>` element, efficiently handling both delete and complete actions using `closest()`. CSS enhances usability by visually distinguishing completed tasks with a line-through effect and background color change.

LEVEL – 2 (Problem 4): Mini Product Search App

Scenario:

An e-commerce company needs a mini product search application that filters products dynamically as users type in the search box.

📌 Requirements

- Display product list from a predefined array.
- Filter products dynamically using input event.
- Display “No Results Found” when applicable.

Technical Constraints

- Must use DOM manipulation.
- Event delegation required.
- Code should follow modular structure.

Expectations:

- Real-time filtering.
- Proper state management.

Code:

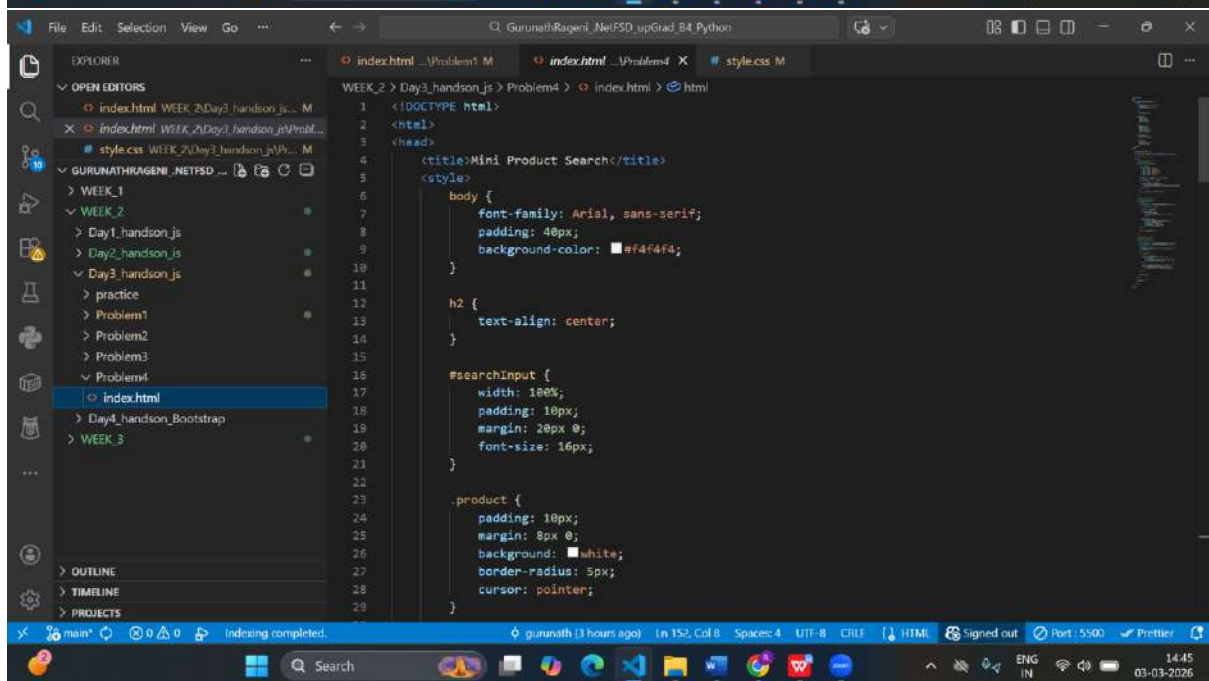
```
File Edit Selection View Go ...  
GurunathRagani_NetPSD_upGrad_B4_Python  
EXPLORER  
OPEN EDITORS  
index.html WEEK_2Day3_handson.js... M  
index.html WEEK_2Day1_handson.jsProbl...  
style.css WEEK_2Day3_handson.jsPr... M  
GURUNATHRAGANI_NETPSD_UPGRAD_B4_PYTHON  
WEEK_1  
WEEK_2  
Day1_handson.js  
Day2_handson.js  
Day3_handson.js  
practice  
Problem1  
Problem2  
Problem3  
Problem4  
index.html  
Day4_handson_Bootstrap  
WEEK_3  
OUTLINE  
TIMELINE  
PROJECTS  
index.html ...Problem1 M index.html ...Problem4 X style.css M  
WEEK_2 > Day3_handson.js > Problem4 > index.html > html > body > script  
2 <html>  
42 <body>  
58 <script>  
115 const App = (() => {  
125 const bindEvents = () => {  
133  
134 // Event Delegation (Single listener for all products)  
135 productList.addEventListener("click", (e) => {  
136 if (e.target.classList.contains("product")) {  
137 alert("Clicked: " + e.target.textContent);  
138 }  
139 });  
140 };  
141  
142 return { init };  
143  
144 })();  
145  
146 // Initialize App  
147 App.init();  
148  
149 </script>  
150  
151 </body>  
152 </html>  
main* 0 0 Indexing completed. Gurunath (3 hours ago) In 145, Col 1 Spaces: 4 UTF-8 CRLF HTML Signed out Port: 5500 Prettier 14:46 03-03-2026
```

```
File Edit Selection View Go ...  
GurunathRagani_NetPSD_upGrad_B4_Python  
EXPLORER  
OPEN EDITORS  
index.html WEEK_2Day3_handson.js... M  
index.html WEEK_2Day1_handson.jsProbl...  
style.css WEEK_2Day3_handson.jsPr... M  
GURUNATHRAGANI_NETPSD_UPGRAD_B4_PYTHON  
WEEK_1  
WEEK_2  
Day1_handson.js  
Day2_handson.js  
Day3_handson.js  
practice  
Problem1  
Problem2  
Problem3  
Problem4  
index.html  
Day4_handson_Bootstrap  
WEEK_3  
OUTLINE  
TIMELINE  
PROJECTS  
index.html ...Problem1 M index.html ...Problem4 X style.css M  
WEEK_2 > Day3_handson.js > Problem4 > index.html > html  
2 <html>  
42 <body>  
58 <script>  
84  
109  
110  
111  
112 /* =====  
113 MODULE: App Controller  
114 ===== */  
115 const App = (() => {  
116  
117 const searchInput = document.getElementById("searchInput");  
118 const productList = document.getElementById("productList");  
119  
120  
121 const init = () => {  
122 UIModule.renderProducts(ProductModule.getProducts());  
123 bindEvents();  
124 };  
125  
126 const bindEvents = () => {  
127  
128 // Real-time filtering  
129 searchInput.addEventListener("input", (e) => {  
130 const filtered =  
131 ProductModule.filterProducts(e.target.value);  
132 UIModule.renderProducts(filtered);  
133 });  
134 }  
135 }  
136 }  
137 }  
138 }  
139 }  
140 }  
141 }  
142 }  
143 }  
144 }  
145 }  
146 }  
147 }  
148 }  
149 }  
150 }  
151 }  
152 }  
153 }  
main* 0 0 Indexing completed. Gurunath (3 hours ago) In 152, Col 8 Spaces: 4 UTF-8 CRLF HTML Signed out Port: 5500 Prettier 14:46 03-03-2026
```

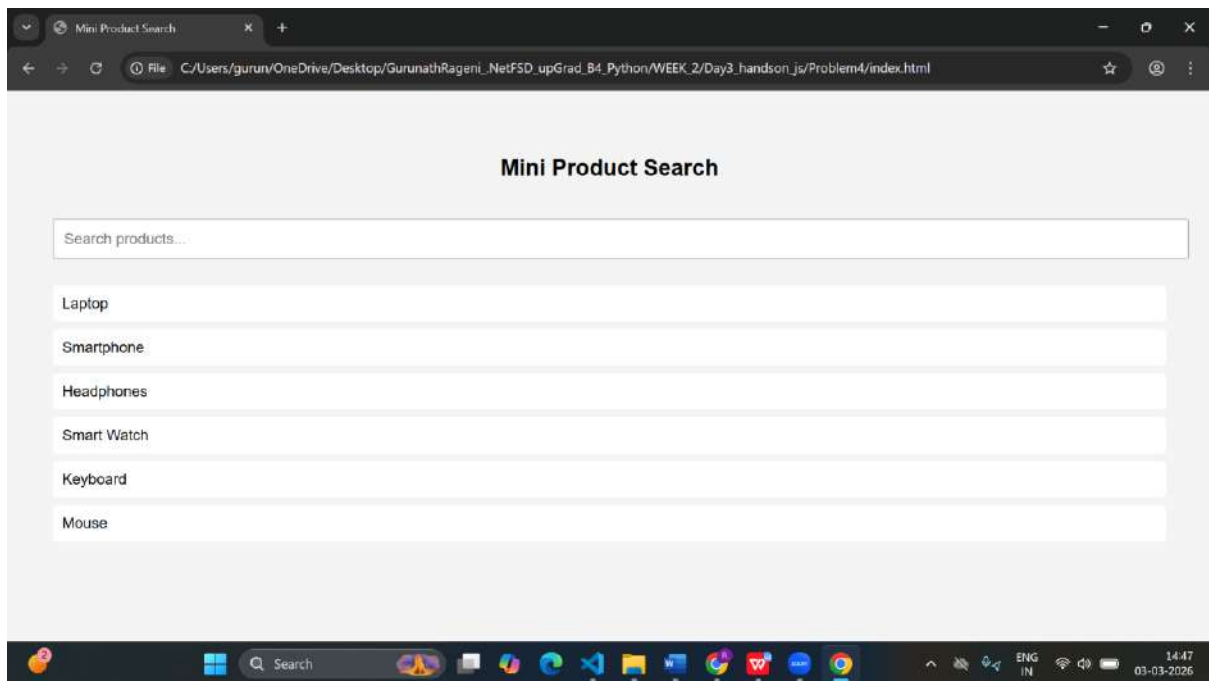


```
2 <html>
42 <body>
58 <script>
81 /* =====
82 MODULE: UI Controller
83 ===== */
84 const UIModule = (() => {
85
86     const productList = document.getElementById("productList");
87
88     const renderProducts = (items) => {
89
90         productList.innerHTML = "";
91
92         if (items.length === 0) {
93             productList.innerHTML =
94                 '<p class="no-result">No Results Found</p>';
95             return;
96         }
97
98         items.forEach(product => {
99             const div = document.createElement("div");
100             div.classList.add("product");
101             div.dataset.id = product.id;
102             div.textContent = product.name;
103             productList.appendChild(div);
104         });
105     });
106
107 }());
```

```
53
54
55 const ProductModule = (() => {
56
57     const products = [
58         { id: 1, name: "Laptop" },
59         { id: 2, name: "Smartphone" },
60         { id: 3, name: "Headphones" },
61         { id: 4, name: "Smart Watch" },
62         { id: 5, name: "Keyboard" },
63         { id: 6, name: "Mouse" }
64     ];
65
66     let state = [...products]; // State Management
67
68     const getProducts = () => state;
69
70     const filterProducts = (keyword) => {
71         state = products.filter(product =>
72             product.name.toLowerCase().includes(keyword.toLowerCase())
73         );
74         return state;
75     };
76
77     return { getProducts, filterProducts };
78 }());
```



Output Screenshot:



Explanation:

This application follows the Module Pattern using IIFEs to separate concerns into ProductModule (data/state management), UIModule (UI rendering), and App Controller (event handling and initialization). It implements real-time search filtering using the input event and updates the DOM dynamically based on filtered results. State is managed internally within the ProductModule, ensuring encapsulation and controlled data access through exposed methods. Event delegation is used on the product list container to efficiently handle click events for dynamically rendered elements.